

The above example is always returning true, this guard is not protecting anything. We are using string token, which works fine but OpaqueToken can be used to not run into name collisions.

Once the token is registered with guard, we can use it in route configuration. CanAlwaysActivateGuard is attached with the route configuration, which gets executed when routing to specific route.

```
• export const AppRoutes:RouterConfig = [
•   {
•     path: '/',
•     component: SomeComponent,
•     canActivate: ['CanAlwaysActivateGuard']
•   }
• ];
```

Define list of guard tokens that can be called so that multiple guards are protecting single route. Guards are executed in a flow defined on the route.

## Defining Guards as Classes

In some scenarios guard needs dependency injection capabilities, as dependencies can be simply injected. For example, AuthService is injected to determine if the user id is authenticated or not.

While creating a guard class, we implement either the CanActivate, CanDeactivate, or CanActivateChild interface, which requires us to have a method canActivate(), canActivateChild(), or canDeactivate() respectively.

```
• import { Injectable } from '@angular/core';
• import { CanActivate } from '@angular/router';
• import { AuthService } from './auth.service';
•
• @Injectable()
• export class CanActivateViaAuthGuard implements CanActivate {
•   constructor(private authService: AuthService) {}
•   canActivate() {
•     return this.authService.isLoggedIn();
•   }
• }
```

An injected class with a canActivate() method has now access to injected dependencies. That method is called when guard is implemented as a class. And can then be used on a route. **d**